Character Set

Definition

A **character set** is the collection of **all valid characters** that a C program can recognize and use. Every C program is built entirely out of these characters — digits, letters, special characters, and white space.

The 4 Categories of Characters in C

DIGITS	LETTERS	SPECIAL CHARACTERS	WHITE SPACE
• Numbers: **0 to 9** Used in constants, numbers, array sizes, etc.	• Uppercase: **A to Z** • Lowercase: **a to z** Used in variables, keywords, etc.	`+ − * / %` → Arithmetic `= ==` → Comparison/assignment `& | ^ ~` → Logical/bitwise `; : ( ) [ ]` → Structure & separation	• Space • Tab (`\t`) • Newline (`\n`) **Use:** separates tokens, improves readability, ignored by compiler (except inside string literals)

Note

- A character set forms the **raw alphabet** of C — every keyword, variable name, and symbol you ever write is built from these 4 categories.
- White space is normally **ignored** by the compiler, but inside a string literal (`"like this"`) it is treated as meaningful text, not ignored.

Fun Tip

Think of the character set as C's **full alphabet + punctuation box** — nothing outside it can be typed into valid C code!

Tokens in C

Definition

In a passage of English text, individual words and punctuation marks are called **tokens**. Similarly, in a C program, the **smallest individual units** that the compiler recognizes are known as **C Tokens**.

Just like a sentence is built from words, a C program is built entirely from tokens — the compiler breaks your code into these pieces before it can understand or execute it.

Types of Tokens in C — 6 Types

6 Types of C Tokens

Keywords | **Identifiers** | **Constants** | **Strings** | **Operators** | **Special Characters**

Quick Preview of Each Type

Token Type	What It Is	Example
Keywords	Reserved words with a fixed, predefined meaning	`int`, `if`, `return`
Identifiers	Names created by the programmer	`marks`, `main`
Constants	Fixed values that never change during execution	`10`, `3.14`, `'A'`
Strings	A sequence of characters inside double quotes	`"Hello"`
Operators	Symbols that perform mathematical/logical actions	`+`, `>`, `&&`
Special Characters	Symbols that define structure and separate statements	`{ }`, `;`, `( )`

Each of these is covered in full detail on the following pages.

Note

- Every single character you type in a C program belongs to **exactly one** of these 6 token categories — nothing falls outside this classification.
- The compiler's very first job (called **lexical analysis**) is to split your raw code into this stream of tokens before checking grammar or meaning.

Fun Tip

Tokens are to C what **words are to a sentence** — the smallest meaningful pieces!

Keywords, Constants, Strings & Special Symbols

KEYWORDS	CONSTANTS	{ } STRINGS	SPECIAL SYMBOLS

Reserved Words
Predefined words with a fixed meaning, reserved by C — **cannot** be used as variable names.
 	
`int, float, char, double, if, else, for, while, return, break, continue`
	Fixed Values
	Do not change during program execution.
	
	Numeric: `10, -25, 3.14`
	Character: `'A', 'z'`
	String Constant: `"Hello"`
		A **sequence of characters** inside double quotes.
		
		`"Hello"`
		`"C Language"`
		`"12345"`
			Characters that define **structure** & **separate statements**.
			
			{ } → block of code
			; → end of statement
			( ) → function calls

Why Each of These Matters

Type	Why It's Needed
Keywords	Give the compiler its fixed vocabulary — without them, C wouldn't know what `if` or `return` means
Constants	Represent values that must stay the same throughout the program (e.g. a fixed tax rate)
Strings	Let a program store and display text (names, messages, sentences)
Special Symbols	Hold the program's structure together — without `{ }` and `;`, C wouldn't know where one instruction ends and another begins

Fun Tip

Keywords = C's **fixed vocabulary** 🔒, Constants = C's **unchanging facts** 📌 — both are locked, unlike variables!

Identifiers & Naming Rules

Definition

An **identifier** is a **name created by the programmer** to identify variables, functions, arrays, structures, and other program elements.

```c
int marks, total_sum;
void main();
char studentName[20];
```

`marks`, `total_sum`, `main`, and `studentName` are all **identifiers** — names the programmer chose.

Rules of Identifier Naming in C

- ✓ **Allowed characters** → only letters (A–Z, a–z), digits (0–9), and underscore _
  `student1, total_marks, sum_2`
- ✗ **Must start with a letter or underscore** → cannot begin with a digit
  ✓ `value1, _value` | ✗ `1value`
- 🅰 **Case-sensitive** → `Name` and `name` are treated as two **completely different** identifiers
- ✗ **No special characters or spaces** → symbols such as `@ $ %` and spaces are not allowed
  ✗ `total%marks` ✗ `my name`
- ✓ **Cannot use keywords** (reserved words) → e.g. `int`, `float`, `return` can never be identifiers
- 📏 **Length** → no strict limit in C, but most compilers only recognize the **first 31 characters** (ANSI standard)
- ✓ **Should be meaningful** (best practice) → improves readability
  `average_score, total_sum – better than: a1, xx`

Note

- An identifier's job is purely to give a **human-readable label** to something the compiler will track by address internally.
- Good identifier names are one of the easiest ways to make your code **self-explanatory** — always prefer `totalMarks` over `tm`.

Fun Tip

Identifiers are like **name tags** 🏷 you create yourself – just follow C's dress code for names!

+ Operators — Overview of All 8 Types

📘 Definition

Operators are symbols that tell the computer to perform some **mathematical or logical function** on one or more values (called operands).

💡 The 8 Types of Operators in C

#	Operator Type	Symbols
1	**Arithmetic Operator**	`+ - * / %`
2	**Relational Operator**	`== != <= >= < >`
3	**Assignment Operator**	`= += -= *= /=`
4	**Logical Operator**	`&&` (AND), `||` (OR), `!` (NOT)
5	**Increment / Decrement Operator**	`++ --`
6	**Conditional Operator**	`? :`
7	**Bitwise Operator**	`& | ^`
8	**Special Operator**	`&` `*` (address-of / pointer)

🧠 One-Line Purpose of Each

Type	Purpose
Arithmetic	Basic math: add, subtract, multiply, divide, remainder
Relational	Compares two values → gives true(1)/false(0)
Assignment	Stores a value into a variable (plain or combined with math)
Logical	Combines multiple true/false conditions
Increment/Decrement	Increases or decreases a variable by 1
Conditional (Ternary)	Shortcut for a simple if-else, in one expression
Bitwise	Works directly on the individual bits of a value
Special	`&` gets a variable's address; `*` dereferences a pointer

📝 Note

- These 8 categories cover **every symbol-based operation** in C — from simple math to advanced pointer work.
- Several of these (Logical, Assignment, Increment/Decrement, Conditional) were already covered in detail with full examples in earlier notes — this page is your **quick-reference summary** of all 8 together.
- The **Special Operator** category (&, *) is the foundation of **Pointers**, covered in full in the Pointers notes.

✨ Fun Tip

`8 operator types = 8 different` **`tools in your C toolbox`** 🧰 `— each built for a different job!`

🏷 Datatypes in C — Overview

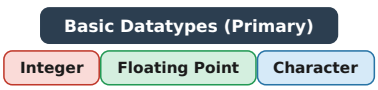
📘 Definition

A **datatype** tells the **type of data** that a variable can store — like a number, a character, or something more complex.

💡 3 Categories of Datatypes in C

Category	Includes	Description
1. Basic / Primary	`int` , `char` , `float` , `double`	C's built-in, fundamental types — every other type is built from these
2. Derived	`array` , `pointer` , `structure` , `union`	Built by combining or extending the primary types
3. User-defined	`typedef` , `enum` , `struct`	Custom types the programmer defines for their own use

🌪 Primary (Basic) Datatypes — Full Breakdown



Branch	Sub-types
Integer → Signed	`int` , `short int` , `long int`
Integer → Unsigned	`unsigned int` , `unsigned short int` , `unsigned long int`
Floating Point	`float` , `double` , `long double`
Character	`char` , `signed char` , `unsigned char`

☞ The full details — explanation, significance, and a working example for **each individual sub-type** — are covered on the next page.

📝 Note

- **Signed** types can store both positive and negative values; **unsigned** types store only zero and positive values, in exchange for a larger positive range.
- Choosing the right datatype affects both **memory usage** and the **range of values** your variable can safely hold.

✨ Fun Tip

`Basic → Derived → User-defined = C's datatypes get` **`more custom-built`** `at each level!` 🚀

🔢 Integer Types — Full Detail

💡 All 6 Integer Sub-types

`int` (Integer)

- **Explanation:** Stores whole numbers (positive & negative).
- **Significance:** Most commonly used for counting, indexing, etc.
- **Example:** `int age = 20;`

`unsigned int`

- **Explanation:** Stores only non-negative whole numbers (0 and above).
- **Significance:** Useful when negative values aren't needed — gives a larger positive range.
- **Example:** `unsigned int population = 50000;`

`short int` (Short Integer)

- **Explanation:** Smaller range integer (uses less memory).
- **Significance:** Saves memory when values are known to be small.
- **Example:** `short int temp = -120;`

`unsigned short int`

- **Explanation:** Only positive short integers.
- **Significance:** More positive range packed into small memory.
- **Example:** `unsigned short int marks = 95;`

`long int` (Long Integer)

- **Explanation:** Larger range integer than normal `int`.
- **Significance:** For big values like population, distance, etc.
- **Example:** `long int distance = 123456789;`

`unsigned long int`

- **Explanation:** Only positive long integers.
- **Significance:** Widest range for very large positive values.
- **Example:** `unsigned long int stars = 400000000;`

✨ Fun Tip

`short` → `int` → `long` = small room, normal room, big warehouse 🏠🏢🏭 — pick the size you actually need!

🔤 Character Types, Floating Types & `void`

💡 Character Types

signed char	unsigned char
Explanation: Stores characters or small integers (can be negative). **Significance:** Represents characters with ASCII codes (-128 to 127). **Example:** `signed char grade = 'A';`	**Explanation:** Stores characters or small integers (0 to 255). **Significance:** Used for binary data, images, ASCII text. **Example:** `unsigned char letter = 'B';`

💡 Floating Point Types

`float`

- **Explanation:** Stores decimal numbers (single precision).
- **Significance:** For storing fractional values like measurements.
- **Example:** `float pi = 3.14;`

`double`

- **Explanation:** Decimal numbers with higher precision (double precision).
- **Significance:** More accurate calculations than `float`.
- **Example:** `double gravity = 9.80665;`

`long double`

- **Explanation:** Even higher precision decimal numbers.
- **Significance:** Used in scientific and financial calculations.
- **Example:** `long double atom_mass = 1.23456789012345;`

💡 The `void` Type

`void`

- **Explanation:** Represents "no value at all."
- **Significance:** Used for functions that don't return anything.

💻 Example

```c
void greet() {
    printf("Hello, World!");
}
```

✨ Fun Tip

`void` = "nothing to return" ∅ — like a function that just *does* something instead of *giving back* something!